

Pitch to Postgres: Designing and Querying a Relational Database for FIFA World Cup Data

Guide: loading the database and running queries on the course PostgreSQL server

Group: Devanshi Dudhatra (202618027), Pujita Sunapu (202618003), Rahul Saha (202618037) **Course:** DS604 Introduction to Data Management **Data:** The Fjelstul World Cup Database by Joshua C. Fjelstul, <https://github.com/jfjelstul/worldcup>, CC-BY-SA 4.0

Everything in this project was built and tested on a local PostgreSQL database. The same code runs on the course server (10.100.71.21). **The only thing you change is the `.env` file.** No code, SQL or notebook needs editing.

Status of the files this guide uses. Steps 1-6 work now (`sql/dd1.sql` from Milestone 2, `notebooks/03_load_data.ipynb` from Milestone 3). Steps 7-8 use files from later milestones: `notebooks/04_queries.ipynb` and `sql/queries.sql` (Milestone 4) and `sql/views.sql` (Milestone 5). This guide will be updated as each one is finished.

Before you start: checklist

You need	Where it comes from
A connection to the university network (on campus, or the university VPN if one is provided)	University IT
Server address: 10.100.71.21	Course
Port, database name, username and password	Instructor or TA (do not guess them)
Whether you must use a particular schema (for example, your own schema in a shared database)	Instructor or TA
Python 3.10 or later and this project folder	Your laptop

Safety rules for the course server

- Never put the password in a notebook, a script or a screenshot. It belongs only in `.env`, which is excluded from git by `.gitignore`.
- Do **not** run `DROP`, `TRUNCATE` or `DELETE` on the course server unless the instructor allows it. The project's reset steps are meant for the local test database only.
- If the server already has tables with our names in your schema, stop and check with the group before doing anything. Step 4 tells you whether such tables exist.

Step 1: Connect to the university network

1. Connect to the campus network (or the university VPN).
2. Check that the server can be reached. Open **PowerShell** and run the following, replacing 5432 with the port you were given:

```
powershell Test-NetConnection 10.100.71.21 -Port 5432
```

You want `TcpTestSucceeded : True`. If it says `False`, you are not on the right network or the port is different. Sort this out before going further.

Step 2: Set up Python (first time only)

Open a terminal in the project folder (`DBMS Project`) and run:

```
python -m pip install -r requirements.txt
```

This installs pandas, SQLAlchemy, the PostgreSQL driver (psycopg2), python-dotenv and Jupyter.

Step 3: Point the project at the course server

1. If you do not have a `.env` file yet, copy the template:

```
powershell Copy-Item .env.example .env
```

2. Open `.env` in a text editor and fill in the course server values:

```
ini DB_HOST=10.100.71.21 DB_PORT=5432 # the port you were given DB_NAME=your_database_name # given by
the instructor DB_USER=your_username # given by the instructor DB_PASSWORD=your_password # given by the
instructor DB_SCHEMA= # leave empty unless told to use a schema
```

- Keep your local settings as comments in the same file, so you can switch back by swapping which lines are commented.
- Passwords containing special characters (`@` : / `#`) are fine; the code escapes them.
- If you were given a schema, write its name in `DB_SCHEMA`. Every table will then be created and queried in that schema.

Step 4: Test the connection (read-only)

```
python src/check_connection.py
```

This script only reads; it changes nothing. A successful run looks like this:

```
Target: your_username@10.100.71.21:5432/your_database_name schema: (default search_path)
server version           PostgreSQL ...
connected as             your_username
database                 your_database_name
search_path              "$user", public
default schema           public
can create tables in that schema True
World Cup tables already present 0
Connection OK.
```

Check two values in the output before continuing:

- **can create tables in that schema** must be `True`. If it is `False`, ask the instructor which schema you may use, put it in `DB_SCHEMA`, and run the check again.

- **World Cup tables already present** should be 0 for a first load. If it is more than 0, someone in the group has already loaded the data. Agree in the group before changing anything.

The password is never printed. If the connection fails, see the troubleshooting section at the end.

Step 5: Create the tables (Milestone 2 file)

The tables must exist **before** loading, because the loader appends rows into existing tables. `sql/ddl.sql` creates 28 tables (the 27 dataset tables plus a small `positions` lookup) and 21 indexes. It contains **no DROP, TRUNCATE or DELETE**, and running it twice is harmless. Use any one of these methods.

Quickest: the project script.

```
python src/apply_ddl.py
```

It prints the target (without the password), runs `sql/ddl.sql`, and ends with `Base tables in schema now: 28`. Never use its `--reset-local` option on the server; the script refuses it anyway when the host is not local.

Option A: from the load notebook. The first cells of `notebooks/03_load_data.ipynb` run `sql/ddl.sql` through the same `.env` connection, so you can go straight to Step 6.

Option B: with psql or pgAdmin.

```
psql -h 10.100.71.21 -p 5432 -U your_username -d your_database_name -f sql/ddl.sql
```

psql asks for the password. In pgAdmin, open the Query Tool on your database, open `sql/ddl.sql` and run it.

If you use a schema and psql, run `SET search_path TO your_schema;` first. The notebook does this automatically from `DB_SCHEMA`.

Step 6: Load the 27 CSV files (Milestone 3 notebook)

1. Start Jupyter from the project folder:

```
powershell python -m jupyter notebook
```

2. Open `notebooks/03_load_data.ipynb` and choose **Kernel > Restart Kernel and Run All Cells**.
3. The notebook: - prints the target as `user@host:port/database`, with no password; - creates the tables from `sql/ddl.sql` (Step 5, Option A); - reads each CSV with `pd.read_csv`, applies the documented cleaning, and loads it with `DataFrame.to_sql(..., if_exists="append", index=False)` in dependency order (parent tables first); - compares the row count of every table with its CSV, then runs 7 integrity checks.
4. **What to check:** - the row-count table shows `OK` for all 27 tables, with 57,269 rows in total; - every integrity check shows `0` problem rows; - no cell shows an error.
5. **Alternative without opening Jupyter:** `python src/run_notebook.py notebooks/03_load_data.ipynb` runs the notebook top to bottom, saves its outputs and writes `notebooks/03_load_data.pdf` for submission.

If something fails during the load (for example, the network drops), nothing is saved: all 27 tables are loaded in **one transaction**, so either everything is committed or nothing is. You can simply run the notebook again.

If the tables already contain data (someone in the group has already loaded it), the notebook stops before loading with the message "These tables already contain data". It never deletes anything on the server. Check the data with `python src/check_connection.py` and agree in the group before changing anything.

Step 7: Run the analytical queries (Milestone 4 notebook)

1. Open `notebooks/04_queries.ipynb` and choose **Kernel > Restart Kernel and Run All Cells**.
2. Each query runs with `pd.read_sql_query` and shows `head(10)`.
3. **Sanity checks** (results that must hold on any correctly loaded database): - Q1: Brazil has 5 titles. - Q10: Miroslav Klose is the top scorer with 16 goals. - Q30 (integrity check) returns **no rows**.

You can also run single queries from `sql/queries.sql` in psql or pgAdmin. Every query has a comment above it explaining what it answers.

Step 8: Create the views (Milestone 5)

Run `sql/views.sql` in pgAdmin or psql, or run the views notebook, which executes it and shows `SELECT * FROM <view> LIMIT 10` for each view. Views store no data, so creating them is safe and quick.

Step 9: Export for submission

In Jupyter choose **File > Save and Export Notebook As > HTML**, or run:

```
python -m jupyter nbconvert --to html notebooks/03_load_data.ipynb
```

Then print the HTML to PDF from your browser. Before submitting, check that no output shows the password. It should not, because the code only ever prints the safe target line.

Switching back to the local database

Edit `.env`: comment out the course server lines and uncomment the local lines (`DB_HOST=localhost` and so on). Nothing else changes.

Troubleshooting

Message	Likely cause	What to do
Missing database settings: ...	<code>.env</code> is missing or incomplete	Do Step 3. Make sure the file is called exactly <code>.env</code> (not <code>.env.txt</code>) and sits in the project folder.
could not connect ... timeout expired or Connection refused	Not on the university network, or wrong port	Step 1. Check the VPN and port.
password authentication failed	Wrong username or password	Re-enter them in <code>.env</code> . Watch for spaces at the end of lines.
database "... " does not exist	Wrong <code>DB_NAME</code>	Ask the instructor for the exact database name.
permission denied for schema public	No <code>CREATE</code> right in <code>public</code>	Ask for a schema you may use and set <code>DB_SCHEMA</code> .
relation "... " does not exist while loading	Tables not created yet, or a different schema	Do Step 5, and check that <code>DB_SCHEMA</code> is the same as when the tables were created.

Message	Likely cause	What to do
duplicate key value violates unique constraint	The data was already loaded (append ran twice)	Stop. See the note in Step 6.
ModuleNotFoundError	Python packages missing	Do Step 2.

Data: Joshua C. Fjelstul, The Fjelstul World Cup Database, <https://github.com/jfjelstul/worldcup>, CC-BY-SA 4.0. No data values were modified.